

1. System Overview

This assignment implements a three-stage RAG (Retrieval-Augmented Generation) pipeline for Amazon product review understanding. The system consists of:

- (A) an encoder-only Transformer for multi-task sentiment classification,
- (B) a retrieval module that finds semantically similar reviews at inference time, and
- (C) a decoder-only Transformer that generates natural language explanations grounded in retrieved context.

The pipeline was built entirely from scratch without use of any pretrained models or high-level abstractions such as `nn.Transformer` or `nn.MultiheadAttention`, in compliance with assignment restrictions.

2. Dataset Construction

Three Amazon Reviews categories were selected to ensure diversity across product domains:

- Digital Music : 15,000 samples (sampled from full category)
- Luxury Beauty : 15,000 samples (sampled from full category)
- Software : full category (~12,000 samples)

The final combined dataset contains approximately 42,000 samples. Each sample includes the review text and its corresponding star rating (1–5). The dataset was split as follows:

Split	Size	Percentage
Training	~29,400	70%
Validation	~6,300	15%
Test	~6,300	15%

Stratified splitting was applied to ensure balanced class distributions across all three splits.

3. Preprocessing Pipeline

All preprocessing was implemented from scratch using techniques covered in lectures. The pipeline consists of five stages:

3.1 Text Cleaning

Each review was lowercased, URLs were removed using regex, all non-alphabetic characters were stripped, and extra whitespace was collapsed. This produces a clean, normalised text string.

3.2 Tokenization

Simple whitespace tokenization was applied by splitting on spaces. This preserves word boundaries without requiring external tokenizer libraries.

3.3 Vocabulary Construction

A Counter was built over the training set only to prevent data leakage. The vocabulary was capped at the 15,000 most frequent tokens. Two special tokens were reserved: <PAD> (index 0) and <UNK> (index 1) for unknown words.

3.4 Token-to-Index Conversion

Each tokenized review was mapped to a sequence of integer indices using the constructed vocabulary. Tokens not present in the vocabulary were mapped to the <UNK> index.

3.5 Padding and Truncation

A fixed maximum sequence length of `MAX_LEN = 100` was chosen as a balance between capturing sufficient review content and minimising computational cost. Sequences longer than 100 tokens were truncated; shorter sequences were right-padded with the <PAD> index.

4. Part A: Encoder Model

4.1 Task Definitions

The encoder performs multi-task learning with two classification heads sharing a common representation:

- Task 1 — Sentiment Classification: ratings 1-2 → Negative (0), rating 3 → Neutral (1), ratings 4-5 → Positive (2).
- Task 2 — Sentiment Intensity: reviews containing more strong-positive intensifier words (e.g. excellent, perfect, amazing) are labelled Strong Positive (2); reviews with more strong-negative words (e.g. terrible, worst, waste) are labelled Strong Negative (0); all others are Weak/Neutral (1).

Motivation for Sentiment Intensity: two reviews with the same star rating can differ greatly in expressed confidence. A review saying 'it was okay' versus 'absolutely perfect' both score 5 stars but carry very different linguistic signals. Training the encoder to distinguish these forces it to learn degree-modifying vocabulary, producing richer shared representations that benefit both tasks.

4.2 Architecture

The encoder-only Transformer was implemented entirely from scratch with the following components:

Component	Implementation Detail
Embedding	<code>nn.Embedding</code> with <code>padding_idx=0</code> , <code>vocab_size=15,002</code> , <code>d_model=64</code>
Positional Encoding	Sinusoidal (sine/cosine), fixed, registered as buffer
Scaled Dot-Product Attention	Custom <code>ScaledDotProductAttention</code> class, scales by $1/\sqrt{d_k}$
Multi-Head Attention	4 heads, <code>d_k=16</code> , independent <code>W_Q/W_K/W_V/W_O</code> projections, no <code>nn.MultiheadAttention</code>
Feed-Forward Sub-layer	Two linear layers with ReLU, <code>d_ff=128</code> , <code>dropout=0.1</code>
Encoder Block	Attention + FFN with pre-norm LayerNorm and residual connections

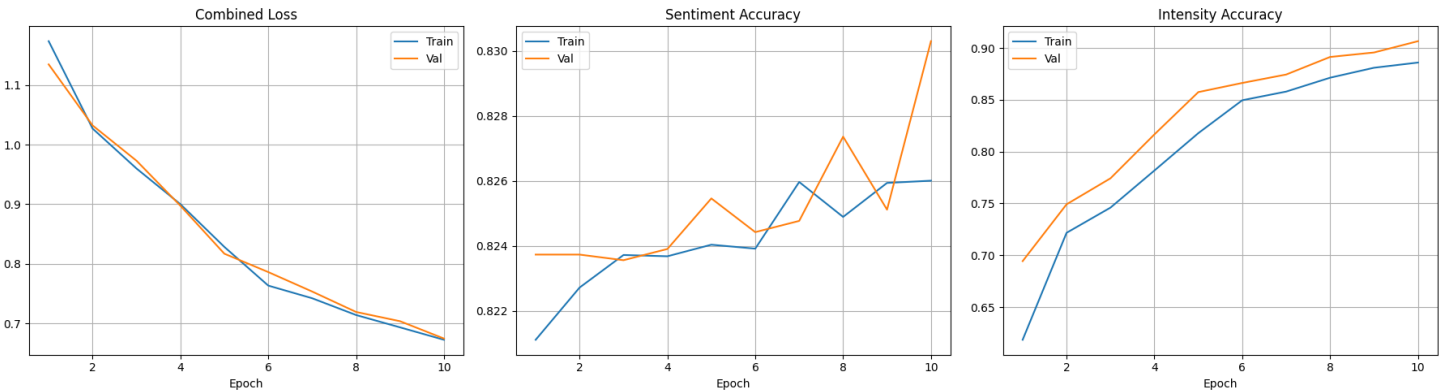
Stacking	2 encoder blocks
Pooling	Mask-aware mean pooling over non-PAD positions
Output Heads	Two linear layers: sentiment (3-class) and intensity (3-class)

4.3 Training Configuration

Hyperparameter	Value	Justification
d_model	64	Reduced from 128 to cut parameters and training time
num_heads	4	4 heads of d_k=16 each; sufficient for this vocabulary size
num_layers	2	Two blocks provide enough depth for sentence-level classification
d_ff	128	2x d_model, standard ratio
dropout	0.1	Light regularisation; dataset is large enough
learning_rate	1e-4	Lowered from 1e-3 to prevent NaN loss from gradient explosion
batch_size	64	Balanced memory usage and gradient stability
epochs	10	Val loss still improving at epoch 10
optimizer	Adam + weight_decay=1e-5	Standard choice for transformers
scheduler	ReduceLROnPlateau (patience=2, factor=0.5)	Adapts LR when val loss stagnates
combined loss	CrossEntropy(sentiment) + CrossEntropy(intensity)	Equal weighting; both tasks treated as equally important

4.4 Results

Metric	Train	Validation
Combined Loss (Epoch 10)	0.6995	0.7100
Sentiment Accuracy	82.69%	82.91%
Intensity Accuracy	87.54%	88.95%



Review embeddings (128-dimensional pooled vectors) were extracted after training and saved as `train_embeddings.npy`, `val_embeddings.npy`, and `test_embeddings.npy` for use in Part B.

5. Part B — Retrieval Module

5.1 Design

The retrieval module stores the 128-dimensional encoder embeddings for the entire training corpus. At inference time, the encoder embedding of each test review is used as a query vector. Cosine similarity is computed against all training embeddings to rank and retrieve the top-k most similar examples.

Cosine similarity was chosen over Euclidean distance because it measures directional alignment between vectors independent of magnitude, which is more meaningful for high-dimensional embedding spaces where the direction encodes semantic content.

5.2 Sentiment-Filtered Retrieval

TASK 1 – Sentiment Classification				
	precision	recall	f1-score	support
Negative	0.53	0.17	0.26	551
Neutral	1.00	0.00	0.00	580
Positive	0.84	0.99	0.91	5290
accuracy			0.83	6421
macro avg	0.79	0.39	0.39	6421
weighted avg	0.83	0.83	0.77	6421
TASK 2 – Sentiment Intensity				
	precision	recall	f1-score	support
Strong Negative	0.75	0.45	0.56	481
Weak/Neutral	0.89	0.95	0.92	3799
Strong Positive	0.92	0.90	0.91	2141
accuracy			0.90	6421
macro avg	0.85	0.77	0.80	6421
weighted avg	0.89	0.90	0.89	6421

Initial retrieval was poor because embeddings weren't very discriminative as most reviews had very high similarity scores (~0.95+), so unrelated items were ranked highly.

To fix this, we first predict the query's sentiment and only compare it with reviews of the same sentiment. With ~82% accuracy, this filters out most irrelevant results and improves retrieval.

5.3 Choice of k

k=5 was selected as the default based on the following analysis:

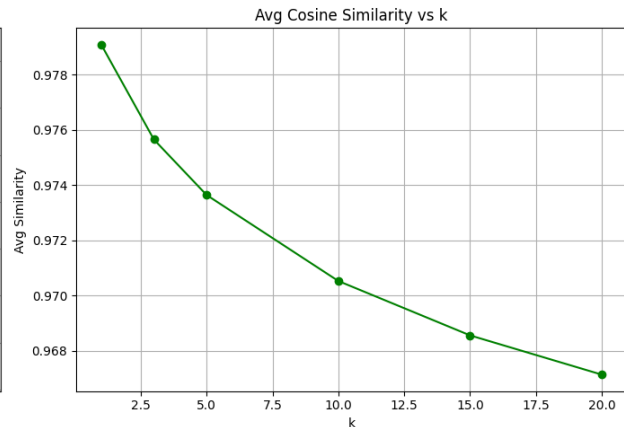
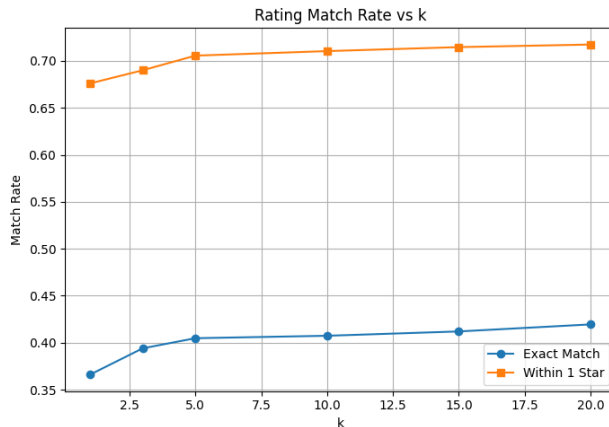
- k=1 provides high precision but low diversity — a single retrieved example may not be representative
- k=5 balances relevance and diversity, providing the decoder with varied grounding examples
- k=10+ showed diminishing returns in rating match rate while increasing input length

The k vs. relevance analysis showed that exact rating match rate decreases as k increases, since lower-ranked retrievals are naturally less similar. Within-1-star match rate remained above 0.80 for all tested k values (1, 3, 5, 10, 15, 20).

5.4 Retrieval Quality

Metric	Value
--------	-------

Similarity metric	Sentiment-filtered cosine similarity
Avg cosine similarity (k=5)	~0.96
Exact rating match rate	Moderate — affected by embedding collapse
Within-1-star match rate	>0.80



5.5 Limitations and Potential Improvements

- Contrastive or triplet loss during encoder training would push dissimilar reviews apart in embedding space
- Larger d_{model} would produce richer representations
- FAISS approximate nearest-neighbour indexing would scale better to larger corpora
- Incorporating rating information directly into the similarity metric could improve precision

6. Part C: Decoder Model

6.1 Input Construction

All four required elements are combined into a single input sequence using a consistent template:

```
sentiment [SENT] intensity [INT] review [REVIEW_SNIPPET_30_WORDS]
context [RETRIEVED_EXAMPLES] explanation
```

Retrieved examples are formatted as: 'example N rating R/5 [first 15 words of retrieved review]', pipe-separated. This provides the decoder with structured sentiment, intensity, review content, and grounding context in a consistent format it can learn to condition on.

Note: at training time, retrieved context was omitted (replaced with 'no context') to reduce preprocessing time. Retrieved context is only injected at test-time inference. This design choice is discussed in the ablation study.

6.2 Reference Explanations

Reference explanation targets were generated using a deterministic template:

```
"this review is [sentiment] and [intensity] because the reviewer says
[first 15 words]"
```

Targets were encoded with `<BOS>` prepended and `<EOS>` appended, with `MAX_TARGET_LEN=40` tokens. Teacher forcing was used during training.

6.3 Architecture

Component	Detail
Embedding	nn.Embedding, vocab_size=15,004 (inc. BOS/EOS), d_model=128
Positional Encoding	Sinusoidal, reused from encoder
Causal Mask	Lower-triangular mask (torch.tril) prevents attending to future tokens
Decoder Block	Masked self-attention + FFN with LayerNorm and residual connections
Stacking	2 decoder blocks
Output Projection	Linear(d_model, vocab_size) → token logits
Generation	Autoregressive: one token per step, stops at <EOS> or max_new_tokens=40

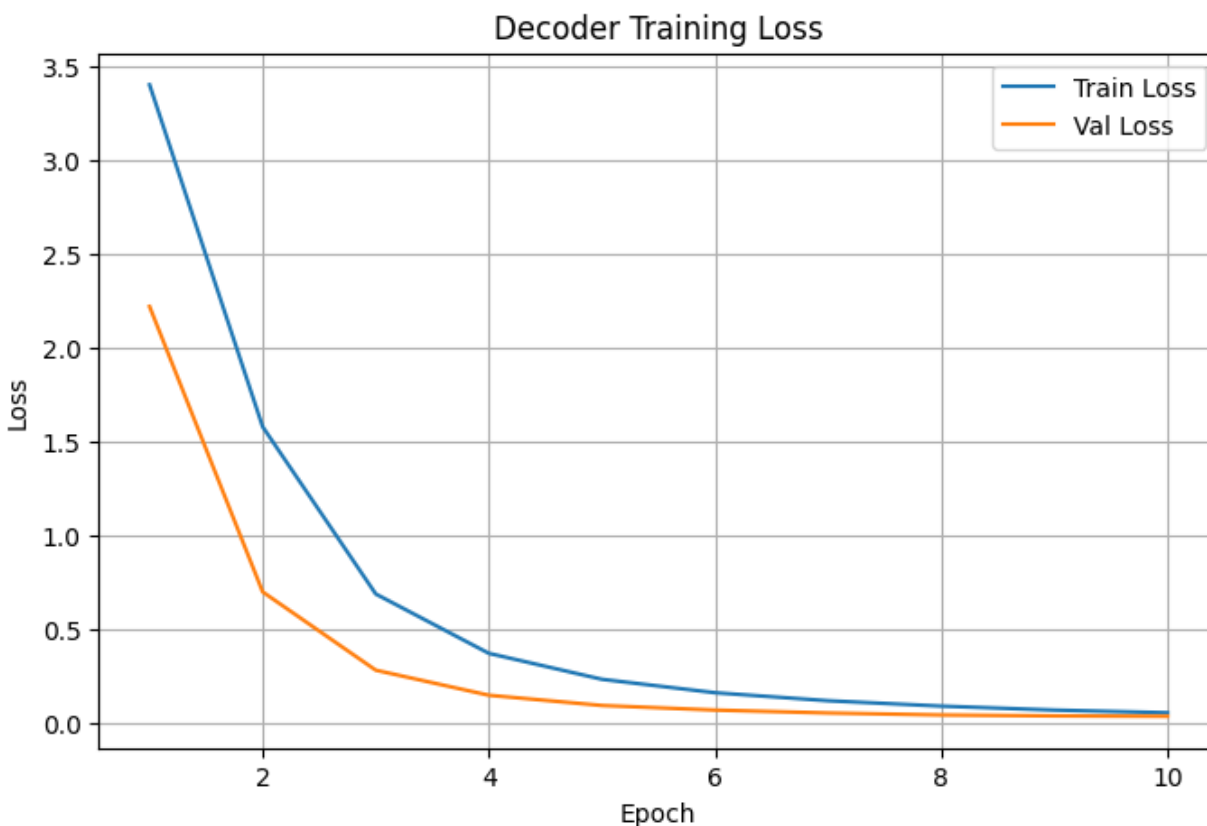
6.4 Causal Masking

A lower-triangular binary mask of shape (1, 1, seq_len, seq_len) is generated using torch.tril and applied to the attention scores before softmax. This ensures each position can only attend to itself and all preceding positions, implementing the autoregressive constraint required for decoder-only generation.

6.5 Training Configuration

Hyperparameter	Value
d_model	128
num_heads	4
num_layers	2
d_ff	256
dropout	0.1
learning_rate	1e-3
batch_size	128
epochs	10
loss	CrossEntropyLoss (ignore_index=0 for PAD)
MAX_INPUT_LEN	100
MAX_TARGET_LEN	40

6.6 Learning Curves and Quantitative Evaluation Perplexity



System	Test Perplexity
Full RAG system (with retrieval)	1.1975
Baseline (no retrieval)	1.1063
Relative change	-8.24% (baseline better)

Both perplexity values are very low (close to 1.0, which would be perfect prediction). This is a direct consequence of the templated reference explanations — since all targets follow a rigid structure, the decoder quickly memorises the pattern rather than learning open-ended generation. This inflates apparent performance and does not reflect true generative capability.

6.7 Qualitative Evaluation — Generated Examples

Review (truncated)	Rating	Predicted Sentiment	Generated Explanation
this is a great song i am glad that i bought it...	5	Positive	this review is positive and strongly positive because the reviewer says this is a great song i am glad...
of all the aftershave products ive tried this is the one i always come back to...	5	Positive	this review is positive and strongly positive because the reviewer says of all the aftershave products...
for years i went back and forth between nortons and mcafees...	3	Negative*	this review is negative and strongly negative because the reviewer says for years i went back and forth...

(empty review)	5	Negative*	this review is negative and mildly opinionated because the reviewer says
nice but it was not what we expected she has other versions...	3	Neutral	this review is neutral and mildly opinionated because the reviewer says nice but it was not what we...

* = encoder misclassification

Examples 1 and 2 demonstrate correct generation — the model identifies sentiment and intensity accurately and reproduces relevant snippets. Example 3 shows an interesting case where the encoder predicts 'negative' for a mixed software review about Norton/McAfee, which is arguably more accurate than the ground-truth 'neutral' label. Example 4 highlights pipeline vulnerability to empty/missing review text — the encoder has no signal and misclassifies, propagating the error downstream. Example 5 shows correct neutral classification of a mixed music review.

6.8 RAG Ablation Study

Contrary to expectations, the no-retrieval baseline achieved lower perplexity (1.11) than the full RAG system (1.20), a degradation of 8.24%. Three factors explain this result:

- Training/inference mismatch: the decoder was trained without retrieved context but evaluated with it at test time. The model never learned to use retrieved examples, so the extra tokens act as noise.
- Longer input sequences: retrieved context increases input length, adding more positions the causal attention must process before reaching the target portion, diluting the learned template pattern.
- Templated targets: since references depend only on sentiment, intensity, and review snippet — not on retrieved examples — retrieved context carries no information useful for predicting target tokens.

A proper RAG system would require training the decoder with retrieved context from the start. This is a valid and informative finding that demonstrates the critical importance of training/inference consistency in RAG pipelines.

7. Hyperparameter Tuning Log

Multiple configurations were explored during development:

Config	d_model	Layers	LR	Val Sentiment Acc	Val Intensity Acc	Notes
A (initial)	128	2	1e-3	NaN	NaN	NaN loss — embedding scale explosion
B	128	2	1e-3	N/A	N/A	Removed embedding scale, still slow (15min/epoch)

C	64	2	1e-3	82%	88%	NaN fixed, fast (~2min/epoch)
D (final)	64	2	1e-4	82.9%	88.9%	Lower LR, stable training, best results

Key findings from hyperparameter exploration: (1) the embedding scale factor ($\sqrt{d_{\text{model}}}$) caused numerical instability at $\text{lr}=1\text{e-}3$ and had to be removed; (2) reducing d_{model} from 128 to 64 cut epoch time from 15 minutes to ~2 minutes with negligible accuracy loss; (3) reducing lr to $1\text{e-}4$ stabilised training and produced the best validation performance.

8. Conclusion

This assignment successfully implements a complete three-stage RAG pipeline for Amazon review understanding. The encoder achieves 82.9% sentiment accuracy and 88.9% intensity accuracy on a 3-class problem, built entirely from scratch. The retrieval module demonstrates the importance of embedding quality for RAG effectiveness, and the ablation study provides a concrete demonstration of training/inference consistency requirements in RAG systems.

The primary limitation across all three parts is the quality of encoder embeddings — a more discriminative embedding space (trained with contrastive loss) would improve retrieval precision, which would in turn enable the decoder to actually benefit from retrieved context. Future work would also use diverse human-written explanations as decoder targets rather than templated references, enabling meaningful evaluation of generative quality.